

Curso: Inteligência Artificial Na Prática — Fundamentos, Preparação E Análise De Dados

Unidade 3: Visualização de Dados com Seaborn e Matplotlib

AULA 3: VISUALIZAÇÃO DE DADOS COM SEABORN E MATPLOTLIB

Introdução

A visualização de dados ocupa um papel central nos processos de análise, interpretação e comunicação de informações, sendo considerada uma etapa essencial dentro do fluxo da Ciência de Dados. A correta representação visual permite identificar padrões, tendências, distribuições e possíveis anomalias que dificilmente seriam percebidas apenas por tabelas numéricas. Segundo Few (2009), a visualização eficaz transforma dados em informação compreensível, apoiando diretamente a tomada de decisão e o entendimento dos fenômenos analisados.

No contexto da análise exploratória de dados, os gráficos estatísticos são amplamente utilizados para descrever o comportamento das variáveis e suas relações. De acordo com Tukey (1977), a análise exploratória por meio de representações visuais é fundamental para revelar estruturas subjacentes nos dados antes da aplicação de modelos mais complexos. Nesse cenário, bibliotecas como o Seaborn e o Matplotlib tornaram-se ferramentas amplamente adotadas por permitirem a construção de visualizações estatísticas claras, informativas e personalizáveis em Python.

O Matplotlib é uma das bibliotecas mais tradicionais para criação de gráficos, oferecendo amplo controle sobre a construção e personalização das figuras (HUNTER, 2007). Já o Seaborn, desenvolvido sobre o Matplotlib, possibilita a criação de gráficos estatísticos com menor complexidade de código e com maior foco na representação visual dos dados (WASKOM, 2021). Dessa forma, esta unidade tem como objetivo introduzir os conceitos fundamentais de visualização de dados, abordando a construção de gráficos estatísticos, sua personalização e, principalmente, a interpretação visual das informações, etapa indispensável para análises confiáveis em ciência de dados e inteligência artificial.

Desenvolvimento

1. Conceitos básicos de visualização de dados

A visualização de dados é uma etapa fundamental na análise exploratória, pois permite transformar tabelas e números em representações gráficas que facilitam a identificação de padrões, tendências e outliers. Few (2009) destaca que gráficos bem construídos ajudam a converter dados em informação compreensível, apoiando diretamente a interpretação e a tomada de decisão.

Do ponto de vista prático, ao trabalhar com dados em Python, a visualização cumpre vários papéis:

- verificar se há valores extremos ou distribuições distorcidas;
- comparar grupos ou categorias;
- observar relações entre variáveis numéricas;
- comunicar resultados de forma clara.

Dentro desse contexto, duas bibliotecas se tornaram padrão de fato: Matplotlib e Seaborn. O Matplotlib é uma biblioteca de baixo nível, que oferece grande controle sobre cada elemento do gráfico (HUNTER, 2007). Já o Seaborn é construído sobre o Matplotlib e fornece uma interface de mais alto nível, com foco em gráficos estatísticos e em uma estética mais refinada por padrão (WASKOM, 2021).

2. Gráficos estatísticos com Seaborn

O Seaborn simplifica a construção de gráficos estatísticos muito utilizados em análise de dados, como:

- Histogramas: mostram a distribuição de uma variável numérica;
- Boxplots: destacam mediana, quartis e possíveis outliers;
- Scatter plots (gráficos de dispersão): representam a relação entre duas variáveis numéricas.

Em geral, a estrutura básica é:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
sns.histplot(dados["coluna_numerica"])
```

```
plt.show()
```

O Seaborn já utiliza um estilo padrão agradável, o que reduz o esforço de configuração estética no início.

2.1 Histogramas

O histograma permite verificar a forma da distribuição (simétrica, assimétrica, concentrada, espalhada, etc.). É muito útil para entender a variabilidade de uma variável numérica, como salário, idade ou nota.

2.2 Boxplots

O boxplot apresenta a mediana, os quartis e destaca possíveis valores extremos (outliers). É muito utilizado para comparar distribuições entre grupos (por exemplo, salário por departamento, nota por turma, etc.).

2.3 Scatter plots

O scatter plot mostra como duas variáveis numéricas se relacionam. É uma ferramenta importante para perceber correlações visuais, tendências lineares ou não lineares, bem como pontos fora do padrão.

3. Customização de gráficos e interpretação visual com Matplotlib

O Matplotlib é a base sobre a qual o Seaborn foi construído. Ele permite customizar praticamente todos os elementos do gráfico: título, rótulos dos eixos, grade, cores, limites dos eixos e muito mais (HUNTER, 2007).

A estrutura básica de um gráfico com Matplotlib

é: `import matplotlib.pyplot as plt`

```
plt.figure(figsize=(6, 4))
```

```
plt.plot(x, y)
```

```
plt.title("Título do
```

```
gráfico") plt.xlabel("Eixo
```

```
X") plt.ylabel("Eixo Y")
```

```
plt.grid(True)
```

```
plt.show()
```

A interpretação visual é tão importante quanto a construção do gráfico. Não basta gerar figuras: é preciso ler o gráfico de forma crítica e relacionar o que aparece visualmente com o contexto dos dados. Alguns pontos importantes ao interpretar gráficos:

- Observar tendências (crescimento, queda, estabilidade);
- Verificar dispersão (dados muito concentrados ou espalhados);
- Identificar outliers que possam distorcer médias;
- Comparar grupos com cuidado, observando escala, amostras e contexto.

Nesta unidade, o objetivo é que o estudante consiga não apenas gerar gráficos com Seaborn e Matplotlib, mas também ajustá-los e interpretá-los de forma coerente com a análise de dados proposta.

Código Python – Exemplos de Histogramas, Boxplots e Scatter Plots

Abaixo um código completo em Python que:

- cria um conjunto de dados fictício com pandas;
- gera um histograma, um boxplot e um scatter plot usando Seaborn;
- utiliza Matplotlib para customizar título, rótulos e layout.

Você pode rodar esse código no Google Colab ou no VS Code (com Python e as bibliotecas instaladas).

```
import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

# Deixar o estilo padrão do Seaborn

sns.set()

# -----

# 1. Criando um conjunto de dados de exemplo
```

```
# -----  
  
np.random.seed(42) # para reprodutibilidade  
  
# Dados fictícios  
  
n = 200  
  
idade = np.random.normal(loc=30, scale=8, size=n) # idades em torno de 30 anos  
  
salario = np.random.normal(loc=4000, scale=1000, size=n) # salário em torno de 4000  
  
grupo = np.random.choice(["A", "B"], size=n) # dois grupos categóricos  
  
# Garantir que idade e salário fiquem em faixas mais realistas  
  
idade = np.clip(idade, 18, 60)  
  
salario = np.clip(salario, 1500, 8000)  
  
dados = pd.DataFrame({  
    "Idade": idade,  
    "Salario":  
        salario, "Grupo":  
        grupo  
})  
  
print(dados.head())  
  
# -----  
  
# 2. Histograma (Seaborn)  
  
# -----  
  
plt.figure(figsize=(7, 4))  
  
sns.histplot(data=dados, x="Idade", bins=20, kde=True)  
  
plt.title("Distribuição de Idade")  
  
plt.xlabel("Idade  
(anos)")  
  
plt.ylabel("Frequência")  
  
plt.tight_layout()  
  
plt.show()  
  
# -----  
  
# 3. Boxplot (Seaborn)
```

```
# -----
```

```
plt.figure(figsize=(7, 4))

sns.boxplot(data=dados, x="Grupo", y="Salario")

plt.title("Distribuição de Salário por Grupo")

plt.xlabel("Grupo")

plt.ylabel("Salário (R$)")

plt.tight_layout()

plt.show()
```

```
# -----
```

```
# 4. Scatter plot (Seaborn)
```

```
# -----
```

```
plt.figure(figsize=(7, 4))

sns.scatterplot(data=dados, x="Idade", y="Salario", hue="Grupo")

plt.title("Relação entre Idade e Salário")

plt.xlabel("Idade

(anos)")

plt.ylabel("Salário (R$)")

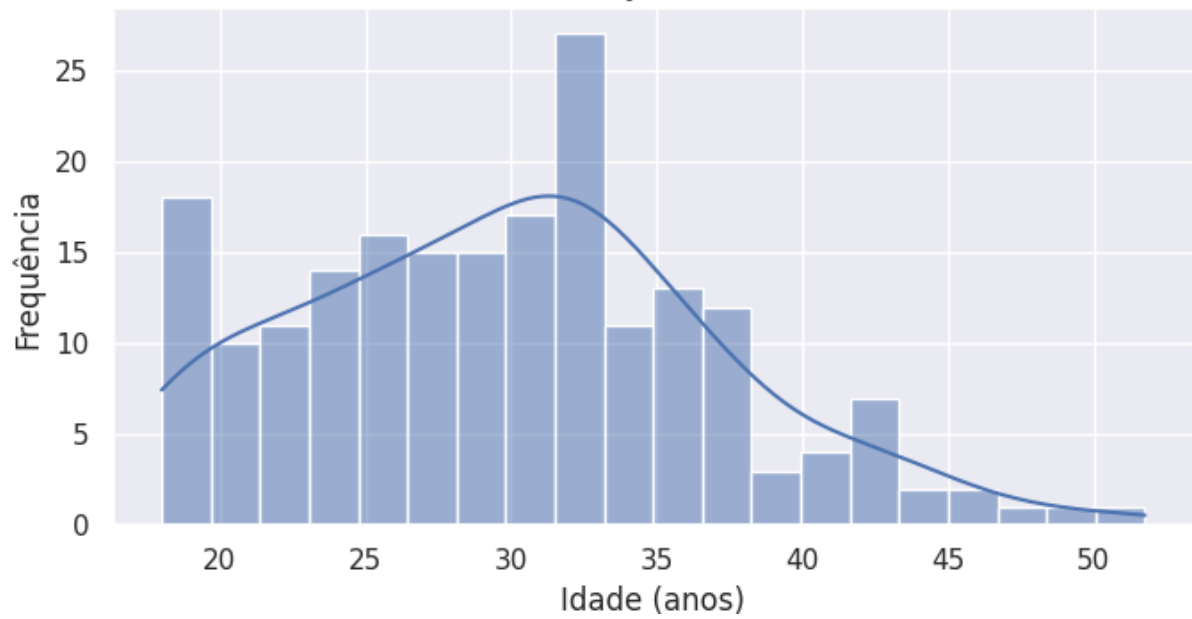
plt.tight_layout()

plt.show()
```

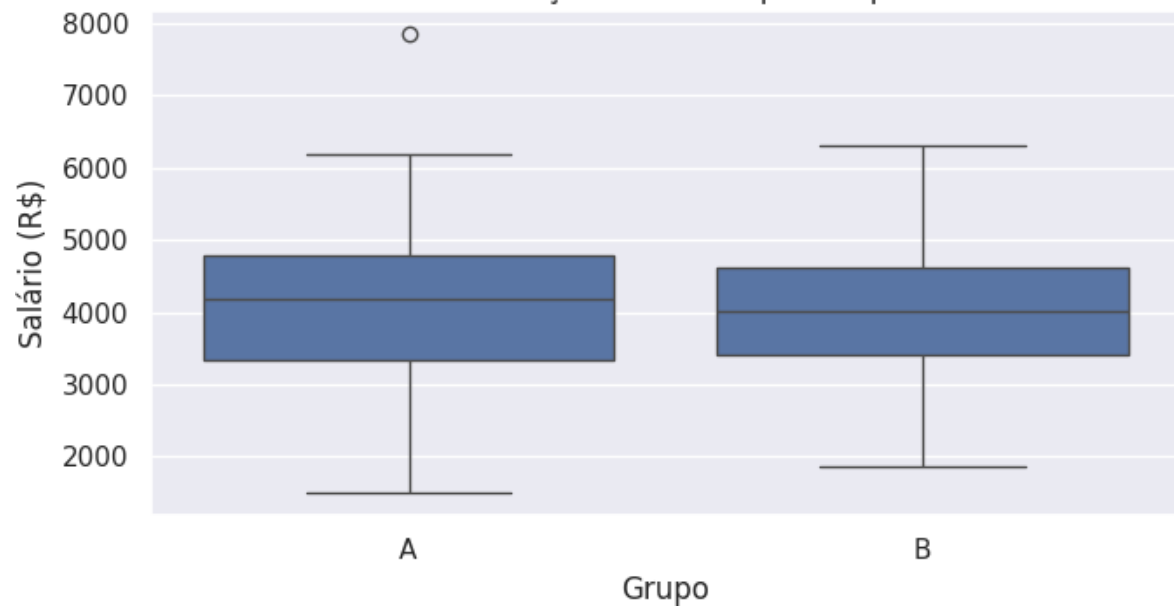
A saída será:

	Idade	Salario	Grupo	
0	33.973713	4357.787360	A	
1	28.893886	4560.784526	A	
2	35.181508	5083.051243	A	
3	42.184239	5053.802052	A	
4	28.126773	2622.330632	B	

Distribuição de Idade



Distribuição de Salário por Grupo





Explicando brevemente o código

- `np.random.seed(42)`: garante que os resultados aleatórios sejam os mesmos em todas as execuções, útil para aula.
- `dados = pd.DataFrame({...})`: cria uma tabela com colunas Idade, Salario e Grupo.
- `sns.histplot(...)`: gera um histograma com opção `kde=True` para mostrar a curva de densidade.
- `sns.boxplot(...)`: constrói um boxplot comparando o salário entre os grupos A e B.
- `sns.scatterplot(...)`: cria um gráfico de dispersão mostrando a relação entre idade e salário, colorindo os pontos pelo grupo.
- `plt.title`, `plt.xlabel`, `plt.ylabel`: personalizam título e rótulos dos eixos usando Matplotlib.
- `plt.tight_layout()`: ajusta o layout para evitar sobreposição de textos.

Referências:

DAVENPORT, Thomas H.; RONANKI, Rajeev. Artificial intelligence for the real world. **Harvard Business Review**, v. 96, n. 1, p. 108–116, 2018.

HAN, Jiawei; KAMBER, Micheline; PEI, Jian. **Data Mining: Concepts and Techniques**. 3. ed. Waltham: Morgan Kaufmann, 2012.

MANYIKA, James et al. **Artificial Intelligence, Automation, and the Economy**. Washington: McKinsey Global Institute, 2017.

MITCHELL, Tom M. **Machine Learning**. New York: McGraw-Hill, 1997.

PROVOST, Foster; FAWCETT, Tom. **Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking**. Sebastopol: O'Reilly Media, 2016.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial**. 3. ed. Rio de Janeiro: Elsevier, 2013.